



RBC: A Randomness-Resistant Block-Grained Compaction Strategy for ZNS SSDs

TIANQI ZHAN, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, wuhan, China

DAN FENG, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, wuhan, China

SHU LI, Alibaba Group, HangZhou, China

ZHENGYONG WANG, Alibaba Group, HangZhou, China

WEI TONG, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, wuhan, China

Research on LSM-tree optimization for ZNS-based SSDs has garnered significant attention. Existing studies primarily focus on reducing garbage collection overhead by ensuring consistent data block lifetimes within a zone. While many works address cascading write amplification between LSM-trees and ZNS SSDs, few evaluate the effectiveness of existing WA reduction strategies on ZNS. Block compaction, a commonly adopted optimization approach, reduces write amplification by shrinking the compaction granularity to the data block level and remapping reusable data blocks. However, Block compaction causes fragmentation in ZNS SSDs, increasing data migration and write amplification during zone reclamation. Moreover, fragmentation reduces the prefetch cache hit rate of ZNS file systems, thereby increasing data block read latency and negatively impacting user read operations and range lookup performance. Additionally, block compaction with a low proportion of reusable data blocks generates more random read operations, leading to higher compaction latency and blocking user read/write operations.

To address these issues, we propose the Randomness-Resistant Block-grained Compaction (RBC) strategy. RBC uses a remapping table to track the location of fragmented data blocks, allowing for more efficient prefetch cache utilization. Furthermore, RBC reduces the data migration overhead caused by fragmentation by delaying the reclamation of highly fragmented zones and migrating remapped data blocks during idle periods. RBC introduces a dynamic compaction strategy that applies block-level compaction for high dirty block ratios and switches to file-level compaction for low dirty block ratios. Additionally, it implements a low-CPU-overhead dirty block detection strategy. Experimental results show that RBC improves throughput by 69% and 37% compared to traditional RocksDB and other block-grained compaction methods, respectively. RBC also improves range query performance by 1.75x compared to other block-grained compaction techniques.

CCS Concepts: • **Information systems** → **DBMS engine architectures**.

Additional Key Words and Phrases: ZNS SSDs, LSM-tree, Block-grained Compaction

Authors' Contact Information: Tianqi Zhan, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, wuhan, hubei, China; e-mail: zhantianqi@hust.edu.cn; Dan Feng, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, wuhan, hubei, China; e-mail: dfeng@hust.edu.cn; Shu Li, Alibaba Group, HangZhou, China; e-mail: s.li@alibaba-inc.com; Zhengyong Wang, Alibaba Group, HangZhou, China; e-mail: zhengyong.wzy@alibaba-inc.com; Wei Tong, Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, wuhan, hubei, China; e-mail: tongwei@hust.edu.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 1544-3973/2025/9-ART

<https://doi.org/10.1145/3764588>

1 Introduction

ZNS SSDs (Zoned Namespace Solid State Drives) are a type of solid-state drive that uses the new NVMe storage interface [6]. ZNS SSDs divide the logical block addresses (LBAs) into equally sized zones (Zones). Each zone allows sequential writes only and can be reused only after the entire zone has been reset. This sequential write requirement enables ZNS SSDs to perform coarser-grained address translation, such as mapping zones to flash blocks, thereby reducing DRAM usage and cutting costs. Additionally, ZNS SSDs offload tasks like garbage collection and data placement to the host, reducing the overhead of garbage collection and allowing users to design technologies that leverage ZNS SSDs more effectively[5].

ZNS SSDs pair naturally with Log-Structured Merge Trees (LSM-trees), which transform random writes into sequential ones to enhance write performance. They are widely used in key-value storage systems, such as RocksDB[7] and LevelDB[14]. The sequential writes in the LSM-tree can lead to data redundancy, necessitating periodic compaction to merge the redundant key-value pairs. [1, 2]. Compaction in LSM-trees introduces high read latency and I/O amplification, with an increase of up to 50 times. [17].

After compaction, many data blocks in the newly created SSTables are redundant, as they are identical to those in the SSTables before compaction. As a result, compaction requires reading and writing a large number of unnecessary data blocks, leading to increased write amplification. To address this, **block-grained compaction** has been proposed. By reducing the granularity of compaction from the file level to the block level, block-grained compaction merges only blocks with overlapping key ranges, while remapping non-overlapping blocks to avoid redundant writes. Simply reducing SSTable sizes would significantly increase compaction scheduling overhead and CPU consumption, leading to user I/O blocking. Block compaction achieves fine-grained optimization and improved throughput while preserving SSTable structure. Figure 1 summarizes key challenges observed from prior work, including findings from the comprehensive survey "Constructing and Analyzing the LSM Compaction Design Space" [24], combined with our own profiling of RocksDB workloads. As shown in the figure 1, compared to KV separation [17] [9], delayed compaction [21], and tiering compaction [8] [22], block-grained compaction performs well in improving write throughput without significantly sacrificing point lookup or tail latency. Block-grained compaction can reduce write amplification by up to 32%. Therefore, it has been widely adopted in several practical storage engines, such as Alibaba's X-engine[15].

However, block-grained compaction introduces challenges when applied to ZNS SSDs. Fragmentation occurs as remapped blocks become scattered across the SSD. Fragmentation critically degrades performance in ZNS SSDs due to: (1) disrupted sequential access patterns that underpin ZNS's bandwidth efficiency, reducing prefetch cache hit rates by 27%; and (2) forced random reads during garbage collection, increasing read latency when reclaiming highly fragmented zones. This fragmentation disrupts sequential read patterns, leading to poor cache performance and higher read latency. This raises the latency for compaction reads and range lookup operations. Additionally, when the proportion of necessary data blocks is high during compaction, block-grained compaction incurs more random read operations. This increases compaction latency and can block user read/write operations.

To address these issues, we propose a Randomness-Resistant Block Compaction (RBC) strategy. RBC introduces a remapping table-based prefetch cache to improve the hit rate for remapped blocks in ZNS SSDs. It also features a reorganization strategy to periodically consolidate remapped blocks, reducing fragmentation. Furthermore, RBC uses a dynamic compaction reading strategy based on the ratio of random reads, optimizing read operations during compaction.

The main contributions of this paper are as follows:

- (1) We propose the RBC strategy, introducing a remapping table-based prefetch cache to improve cache hit rates for remapped blocks in ZNS SSDs.
- (2) We design a reorganization strategy for remapped data blocks to reduce fragmentation in ZNS SSDs.

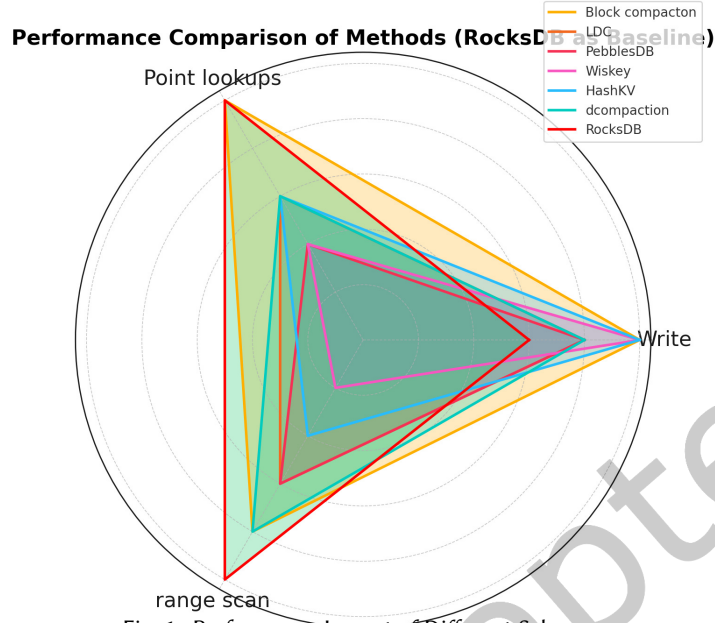


Fig. 1. Performance Impact of Different Schemes.

- (3) We propose a dynamic compaction strategy that adapts granularity based on block dirtiness. Additionally, it implements a low-CPU-overhead dirty block detection strategy.

2 background and motivation

We now present the necessary background of ZNS SSD, LSM-trees and motivation to optimize block compaction. A more detailed survey on LSM-basics can be found in the literature [10, 19].

2.1 ZNS SSD

Zoned Namespace Solid State Drives (ZNS SSDs) introduce a novel storage architecture that partitions capacity into independently managed zones, each requiring sequential writes while permitting random reads. Figure 2 illustrates how block-level compaction in ZenFS leads to logical fragmentation, as multiple updates to the same key result in scattered block placement.

This architecture reduces write amplification (1.8–2.5×), extends device lifespan (>30% P/E reduction), and enables host-side performance tuning. The architecture’s fundamental innovation lies in its zone-oriented access model, which replaces traditional block-level mapping with sequential zone writes, eliminates complex FTL overhead (95% DRAM reduction), and shifts garbage collection control to host software. These characteristics make ZNS SSDs particularly effective for write-sequential workloads like log-structured storage systems, time-series databases, and large-scale analytics pipelines, where their ordered write constraints align naturally with application access patterns. Leveraging orderly writes and zone management, ZNS SSDs improve performance through key features such as:

- **Zone Management:** Storage is segmented into zones, each requiring sequential write operations.
- **Write Pointer:** Independent write pointers for each zone ensure orderly writes and enhance management efficiency.
- **Host-side Garbage Collection:** The host manages garbage collection, reducing the amount of data migration during the process.

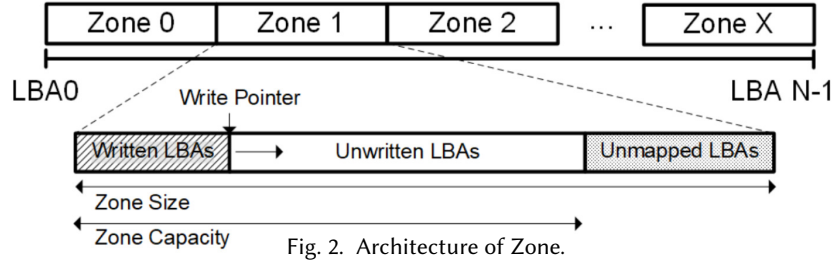


Fig. 2. Architecture of Zone.

While ZNS reduces write amplification via sequential zones, it suffers from read inefficiencies due to fragmentation and limited address flexibility.

2.2 LSM-tree

To support large-scale storage, many KV stores are designed based on the LSM-tree (e.g., [23], [12], [17], [22], [9]). Figure 3 depicts the design of an LSM-tree-based KV store, using RocksDB as an example. An LSM-based KV store maintains in-memory MemTables and immutable backups, along with multi-level on-disk SSTables. SSTables on disk are organized as a hierarchy of multiple levels, each of which contains multiple sorted SSTables and has fixed data capacity. The data capacity in each level increases from lower to higher levels in the LSM-tree. The main feature of the LSM-tree is that data flows only from lower to higher levels and key-value pairs in each level are sorted to balance the trade-off between read and write performance.

The insertion of a new key-value pair works as follows. The new key-value pair is first appended to the write-ahead-log file. The LSM-tree KV store then adds the key-value pairs to the MemTable. Once a MemTable becomes full, it is converted to an Immutable MemTable, which is flushed to level L0 as a SSTable on the disk. The look-up of a key-value pair searches for multiple SSTables. Specifically, LSM-tree based KV store searches all SSTables at the L0 level whose key range contains the key, and one SSTable at each of the other levels whose key range contains the key, until it finds the key-value pair. To reduce read latency, state-of-the-art LSM-engines use in-memory data structures, such as Bloom filters[11] and fence pointers[12].

LSM Metadata Concepts. We briefly define two relevant compaction concepts: (1) Fence Pointers track SSTable key ranges to optimize lookups, and (2) Full vs. Partial Compaction differ in merge scope and overhead tradeoffs [24].

2.3 LSM-Compactions

To amortize data movement, and thus, avoid latency spikes, state-of-the-art LSM-engines organize data into smaller files, and perform compactions at the granularity of files instead of levels. If Level i grows beyond a threshold, a compaction is triggered and one file (or a subset of files) from Level i is chosen to be compacted with files from Level $i + 1$ that have an overlapping key-range. This process is called as partial compaction. Figure 4 presents a comparative illustration of the full compaction and partial compaction routines in LSM-trees. Specifically, RocksDB tolerates overlapping keys at L1 to avoid write stalls [3, 4], exacerbating compaction complexity.

2.4 Block-grained Compaction

While shrinking SSTables increases scheduling overhead, block-grained compaction enables fine-grained merges and preserves scalability. Block compaction reduces write amplification by remapping reused data blocks to avoid redundant reads and writes. During block compaction, data blocks are categorized into **Clean Blocks** and **Dirty Blocks**:

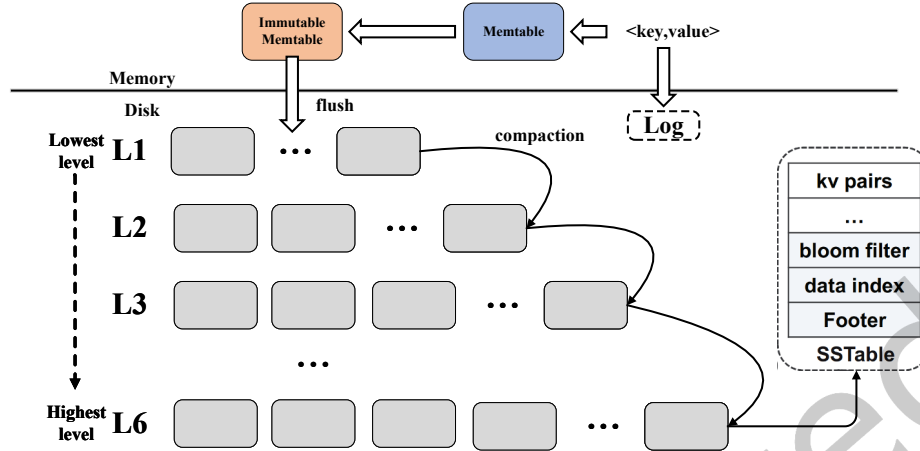
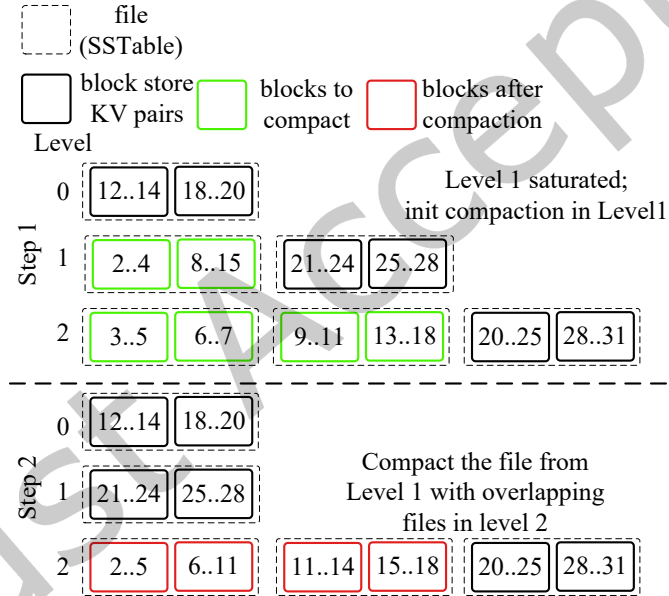


Fig. 3. Architecture of an LSM-tree based KV store (RocksDB).



Compaction in RocksDB: compacts equi-sized files with overlapping files from two consecutive levels

Fig. 4. Compaction: Merging and Sorting KV Pairs.

- **Clean Blocks:** Blocks that do not contain overlapping key-value pairs and thus require no modifications. These blocks can be directly reused.
- **Dirty Blocks:** Blocks that contain overlapping key-value pairs and must be read, merged, sorted, and rewritten.

The process of block-grained compaction is illustrated in the figure 5 .

Classification Phase.

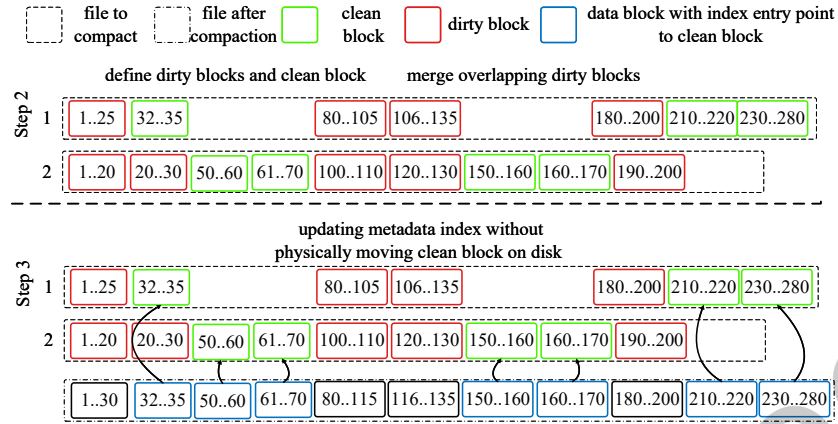


Fig. 5. Block-Grained Compaction: Merging and Sorting Data Block.

- The KV engine scans SSTable files at level i , identifies clean and dirty blocks, and marks the dirty blocks that require block-grained compaction.
- The KV engine reads dirty blocks into memory.

Merge and Rewrite Phase.

- The KV engine merges dirty blocks in memory.
- The KV engine writes the merged key-value pairs into newly generated SSTables.
- By reusing clean blocks, block-grained compaction avoids redundant reads and writes of clean blocks.

Metadata Update Phase.

- Index blocks are reconstructed for all valid data blocks (including clean blocks and newly generated dirty blocks) in the new SSTable.
- Remapping information for reused clean blocks is updated, redirecting their references to the original SSTable.
- Metadata such as index block offsets and Bloom filters is recorded in the tail region.

Alternative Approaches. One possible alternative to achieve finer compaction granularity is to reduce SSTable sizes. However, we find that significantly decreasing SSTable sizes introduces severe drawbacks:

- (1) Compaction scheduling and file prioritization overhead increases dramatically due to the sheer number of small files;
- (2) Frequent small compactions increase CPU consumption and can block user I/O operations;
- (3) Overall compaction efficiency degrades due to fragmented I/O patterns and smaller merge batches.

In contrast, **Block compaction** achieves fine-grained optimization by tracking remapped blocks individually, while preserving the benefits of large SSTables and maintaining compaction scalability.

2.5 Motivation

We modified the firmware on the same SSD to support both the traditional block SSD interface and the ZNS SSD interface. Using YCSB to test a 100GB write workload, we compared the performance of RocksDB with block-grained compaction on a conventional SSD and RocksDB with block-grained compaction on a ZNS SSD. Our findings indicate that block-grained compaction improves throughput on ZNS SSDs by only 11% to 19%, which is significantly lower than the 31% improvement observed on conventional SSDs, as shown in figure 6.

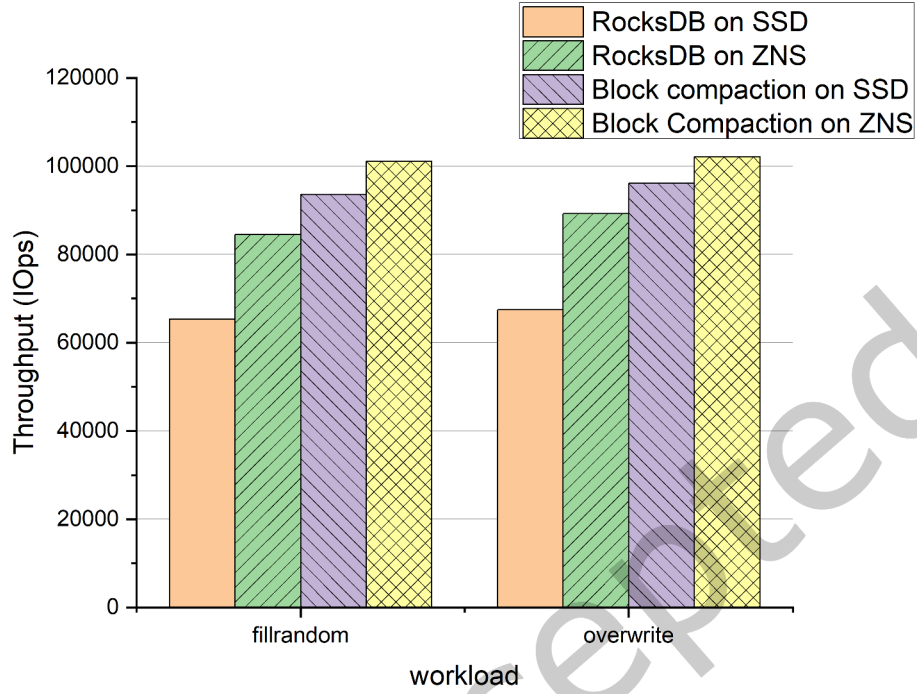


Fig. 6. Write Throughput Comparison of Block-Grained Compaction with RocksDB on SSD and ZNS SSD.

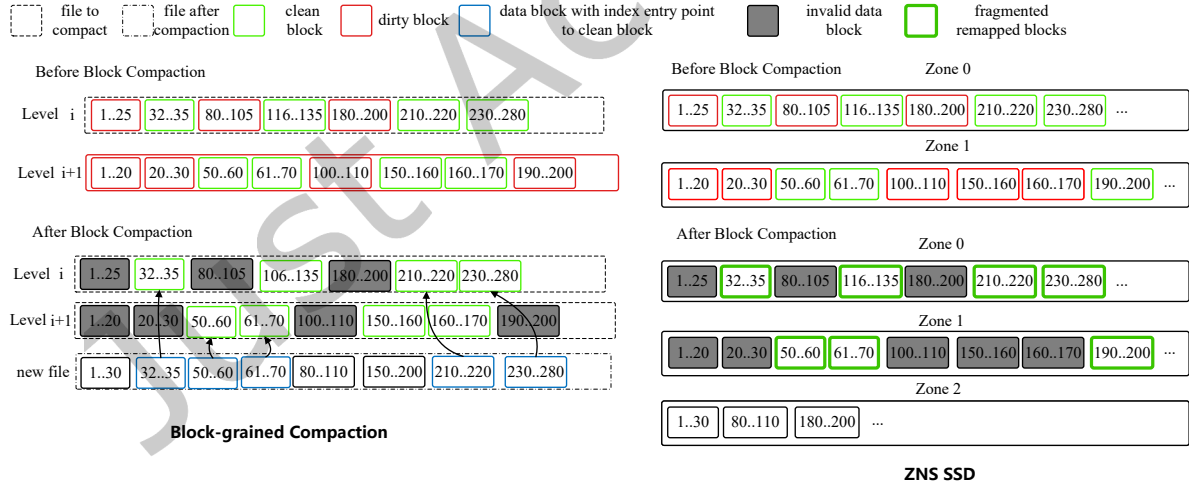


Fig. 7. Block Compaction Causes Fragmented Storage of Data Blocks.

The challenges of optimizing block compaction on ZNS SSDs primarily stem from three issues: **the additional migration of remapped data during zone garbage collection**, **low cache hit rates in ZNS file systems**, and **high random read latency during compaction**. These challenges arise due to the fragmented nature of

data blocks after compaction, which directly impacts the performance of ZNS SSDs, particularly in environments requiring high throughput and low latency.

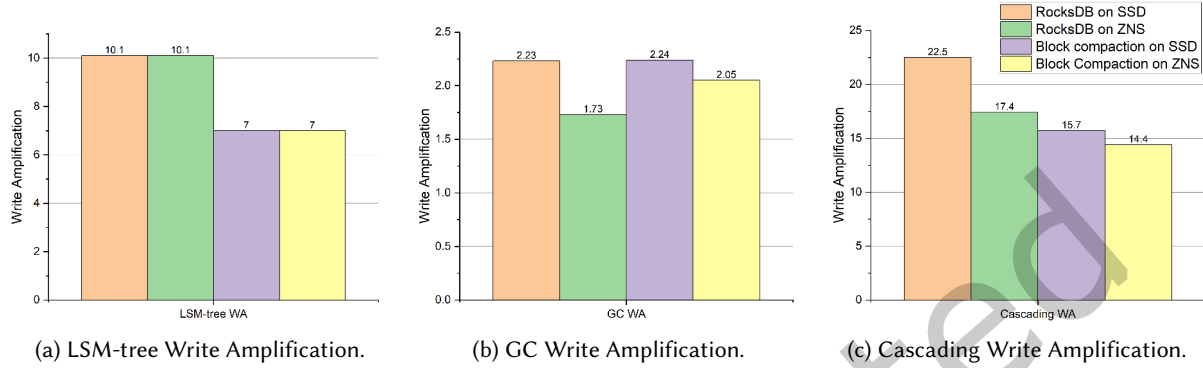


Fig. 8. Comparison of Write Amplification

2.5.1 Fragmentation increases write amplification during Zone garbage collection. We modified the firmware on the same SSD to support both the traditional block interface and the ZNS interface. Using a YCSB workload to random write 100GB of data, we compared the LSM-tree write amplification, ZNS SSD garbage collection (GC) write amplification, and cascading write amplification between RocksDB with block-grained compaction on a traditional SSD and RocksDB with block-grained compaction on a ZNS SSD. As shown in the figure 8, while block compaction reduces LSM-tree write amplification by approximately 30%, it increases ZNS SSD GC write amplification by 18%, leading to only a 9.3% reduction in cascading write amplification.

While block compaction reduces LSM WA, it increases GC WA due to fragmentation, limiting overall amplification improvement. This limits the reduction in cascading write amplification for key-value storage systems, diminishing the performance gains achieved by block compaction in reducing LSM-tree write amplification.

2.5.2 Data Block Fragmentation and ZNS Filesystem Cache Hit Rate. As illustrated in figure 7, block compaction leads to fragmentation of remapped data blocks across the ZNS SSD. This fragmentation prevents the ZNS file system from effectively prefetching logical data block addresses, resulting in a 27% reduction in cache hit rates (figure 9). The reduced cache hit rate significantly increases read block latency, ultimately exacerbating the overall latency of compaction and read operations.

2.5.3 Impact of High Dirty Block Ratio. Unless otherwise noted, we define *dirty blocks* as logical key-value ranges partially invalidated by updates or deletions, and *clean blocks* as fully valid, non-overlapping ranges.

As shown in Figure 10, dirty blocks from different SSTable levels must be merged during compaction. Due to partial invalidation and misalignment, many dirty blocks are scattered across memory. When read requests span such dirty keys, the system must issue multiple non-contiguous I/O operations to reconstruct a logical key range.

This fragmentation breaks spatial locality, reduces prefetch efficiency, and introduces latency overhead during reads. RBC mitigates this issue by reorganizing fragmented dirty blocks and aligning clean blocks through background reorganization.

The ratio of clean to dirty blocks during compaction significantly impacts read performance: as illustrated in Figure 11, a lower reuse ratio (i.e., more dirty blocks) triggers increased random I/O operations, leading to higher read latency. This effect becomes particularly severe when the dirty block ratio exceeds 80%, with average read latencies reaching up to 22ms.

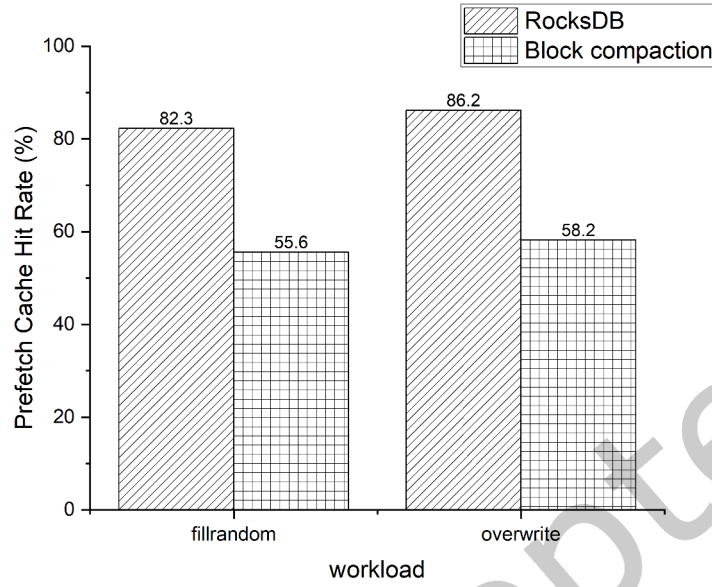


Fig. 9. Prefetch Cache Hit Rate of ZNS File System Under Write Workload.

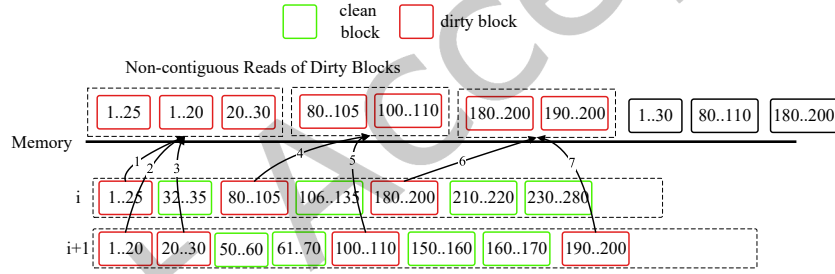


Fig. 10. Non-contiguous Reads of Dirty Blocks.

Note: Dirty blocks denote partially invalidated ranges. As dirty ratio increases, more random reads occur, elevating average read latency up to 22ms.

2.5.4 Research Motivation. Based on the above analysis, the motivation of this study lies in addressing the challenges of low cache hit rates and high compaction random read latency when ZNS SSDs are combined with block-grained compaction. These issues stem from two main factors:

- The ZNS file system's inability to effectively prefetch the logical addresses of remapped data blocks, leading to a decrease in cache hit rates.
- The lack of optimization strategies to handle high dirty block ratios during block-grained compaction, which exacerbates random read overhead.

Given these challenges, we propose a **Randomness-Resistant Block Compaction (RBC)** strategy, which aims to improve the cache hit rate in the ZNS file system and reduce the overhead of random reads during the read process. By optimizing the handling of dirty blocks during block compaction, RBC enhances the performance

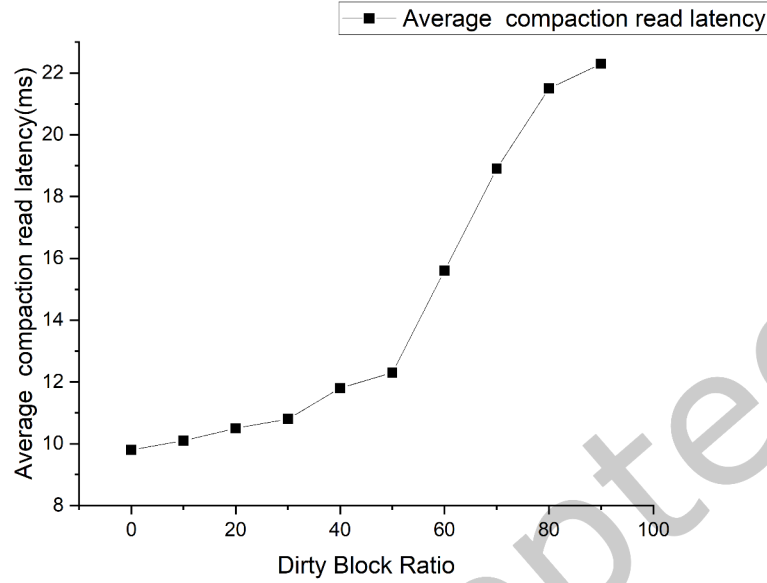


Fig. 11. Relationship Between Average Dirty Block Read Latency and Proportion of Dirty Blocks.

of key-value storage systems on ZNS SSDs, improving throughput and reducing latency for both write and range lookup operations.

3 Randomness-Resistant Block-Grained Compaction Design

3.1 Overview of RBC

In order to address the issues of high read latency and additional random read overhead caused by reading remapped data blocks during the block compaction process of ZNS SSDs, we propose a **Randomness-Resistant Block-Grained Compaction (RBC)** strategy. RBC reduces read latency by integrating spatially guided prefetching, background reorganization, and adaptive read strategies. As shown in figure 12, these modules are:

- (1) **Remapping Table-Based Prefetch Cache:** Reduces high latency when reading remapped data blocks.
- (2) **Remapped Data Block Reorganization Strategy:** Optimizes the storage layout of data blocks to reduce random read I/O.
- (3) **Dynamic compaction Read Strategy Based on Random Read Ratio:** Dynamically adjusts the compaction read strategy based on the reuse ratio during the compaction process to optimize I/O operations.

RBC intercepts read and write I/Os to coordinate remap-aware processing across its components. During a write, the system appends key-value pairs to memtables and performs block-level compaction upon flush, recording new block mappings in the remap table. Updates and reorganizations modify these mappings without rewriting unaffected blocks.

For reads, the remap table is first consulted to resolve the current physical location of the requested data. If remapped, the updated address is returned. The prefetch cache then uses spatial hints from the remap history to fetch adjacent blocks, improving cache hit rate despite fragmentation.

The hybrid compaction engine monitors SSTable fragmentation. It applies block-level compaction to minimize write amplification and triggers background reorganization to restore physical contiguity when fragmentation

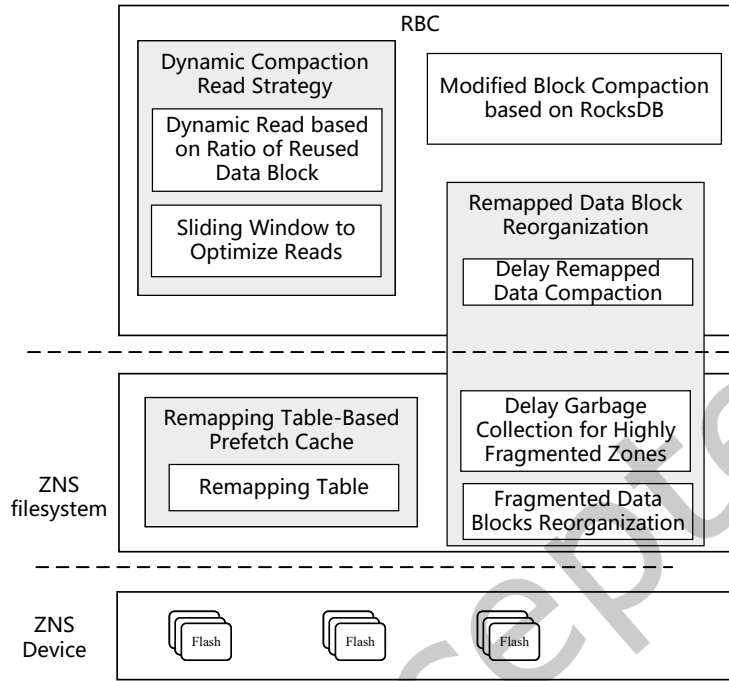


Fig. 12. The RBC Architecture.

exceeds a threshold. These actions are non-blocking and designed to minimize interference with foreground traffic.

3.2 Remapping Table-Based Prefetch Cache

Traditional ZNS file systems are unable to perceive the remapping logical addresses of data blocks. This results in ineffective cache hits when reading remapped data blocks, thereby increasing read latency. This degradation, however, is not an inherent flaw of block-level compaction itself. Rather, it stems from the mismatch between block-granular SSTable layouts and ZenFS's zone-aligned, sequential file abstraction. ZenFS offers limited support for random block updates and lifetime tracking, leading to fragmented SSTables, incorrect zone lifetime hints, and disrupted spatial locality. These limitations result in reduced prefetch hit rates and performance penalties, as seen in Figure 9.

RBC addresses this systemic limitation by introducing an integrated framework that enables block-level compaction to work harmoniously with ZNS SSDs. It coordinates remapping, prefetch prediction, and background reorganization to restore locality and fully realize the potential of fine-grained compaction on modern ZNS devices.

To fundamentally address this limitation, RBC introduces a novel remapping table mechanism that enables precise prefetching of remapped data blocks, significantly improving cache efficiency. With this mapping table, the ZNS file system can identify physically contiguous remapped blocks corresponding to logically related accesses and preload them into the prefetch cache, thereby improving cache hit rates and reducing read latency.

Remapping Table. During block compaction, the system continuously updates the remapping table, as shown in figure 13, which contains the logical and physical addresses of each data block. Each remapping entry

records logical and physical addresses, flags, and sizes, and is dynamically maintained in host memory with crash consistency. RBC stores the remapping table in non-volatile memory to ensure crash consistency.

To ensure durability, all remap table updates are WAL-logged before being applied to NVM-resident metadata. Updates are batched and flushed asynchronously every 1 MB or 100 ms to reduce write amplification while maintaining recovery consistency. Upon crash, RBC replays the WAL to restore the remap table, similar to RocksDB's recovery process. Since the remap metadata is compact (e.g., 180 MB for 400 GB datasets), recovery typically completes within a few seconds. RBC guarantees atomicity of remap operations and ensures all committed keys can be located correctly after a crash.

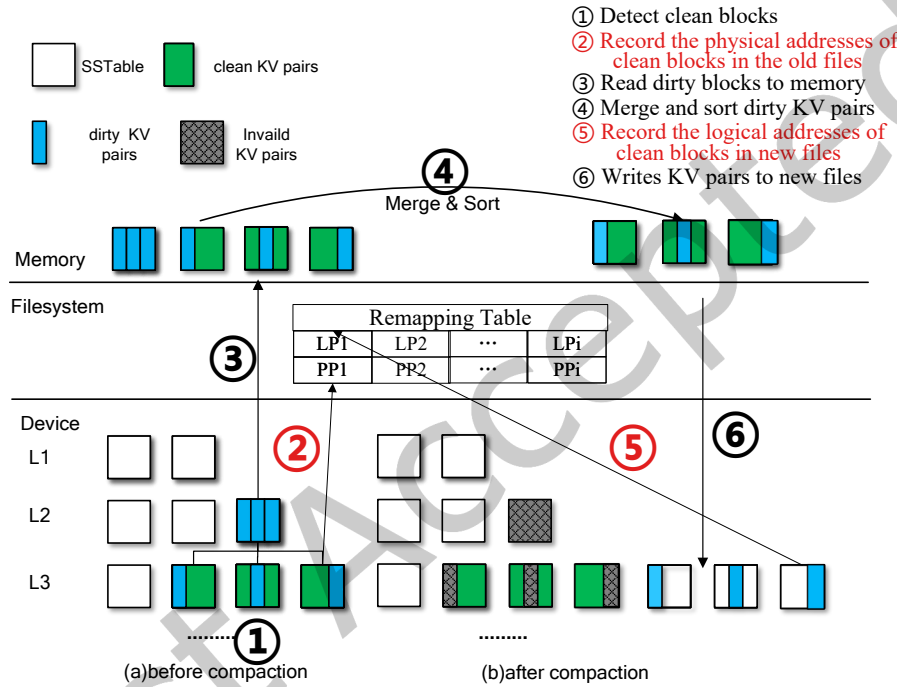


Fig. 13. Write-side remapping procedure.

Unlike static designs, RBC's table employs a scalable hash index with two key innovations:

- **Dynamic Hash Table Expansion:**
 - Employs dynamic hashing technology where bucket count grows linearly with data volume.
 - Each hash bucket maintains an independent persistent log segment to ensure consistency during expansion.
- **Consistency Guarantees:**
 - Ensures crash recovery through Write-Ahead Logging (WAL).

Hash Table Design. RBC uses an extendible hash table to manage block-to-file mappings. The table starts with a fixed number of buckets and dynamically doubles upon hash collisions or load factor exceeding a threshold (e.g., 0.75). Each bucket contains up to 64 entries and is stored in a fixed-size NVM-resident slab. Directory pointers are held in DRAM, with per-bucket locks for concurrent access. To ensure performance under high mutation rates,

remap operations are batched and applied using lock coarsening and pre-allocated entry buffers. This design enables fast remap lookups with minimal contention and a predictable memory footprint (e.g., <0.4 GB for 1 TB data).

Prefetch Operation. When the ZNS file system receives a read request, it checks the remapping table to locate the physical position of the target block.

If logical access patterns suggest access to nearby data, RBC uses the remapping table to locate physically contiguous remapped blocks, even if their logical addresses have been scattered. These blocks are proactively prefetched into the cache, as illustrated in Figure 14.

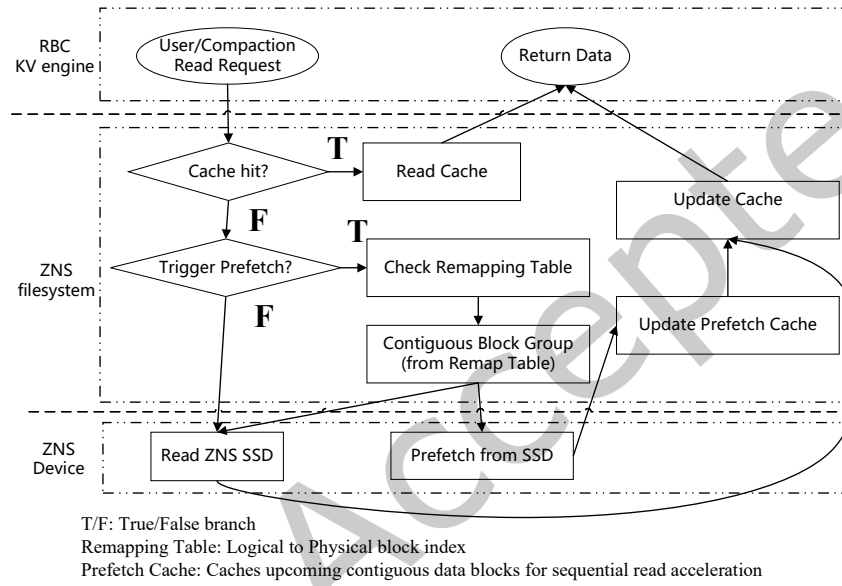


Fig. 14. Read-side Remapping and Prefetching.

Cache Hit Rate Optimization. To improve cache hit rates for remapped data, RBC adopts a *spatially guided prefetching* strategy. This mechanism interprets logical access behavior to locate physically contiguous key-value blocks via the remapping table. By doing so, RBC performs targeted prefetching of these blocks into the prefetch cache, thereby maximizing spatial locality and improving read performance even when logical layouts are fragmented.

Cache Eviction and Management. RBC employs an LRU-based eviction policy for its prefetch cache, with spatial locality grouping to prevent eviction of sequentially adjacent blocks prematurely. The cache size is bounded (e.g., 64 MB), and its occupancy is periodically monitored. Under memory pressure, entries with low access frequency or that belong to reorganized SSTables are evicted first. To avoid cache pollution caused by non-sequential or bursty accesses, RBC tags remapped blocks and limits their reuse priority unless access patterns indicate locality. This policy balances responsiveness with memory efficiency.

Scalable Metadata Management. To ensure scalability under large-scale datasets (e.g., multi-TB), RBC adopts a dynamically expandable remapping table, where only blocks that undergo remapping are tracked. Furthermore, RBC's background reorganization mechanism continuously reduces the number of active remapped blocks over time, effectively bounding the remapping table size. In extreme cases, if memory pressure is detected (e.g.,

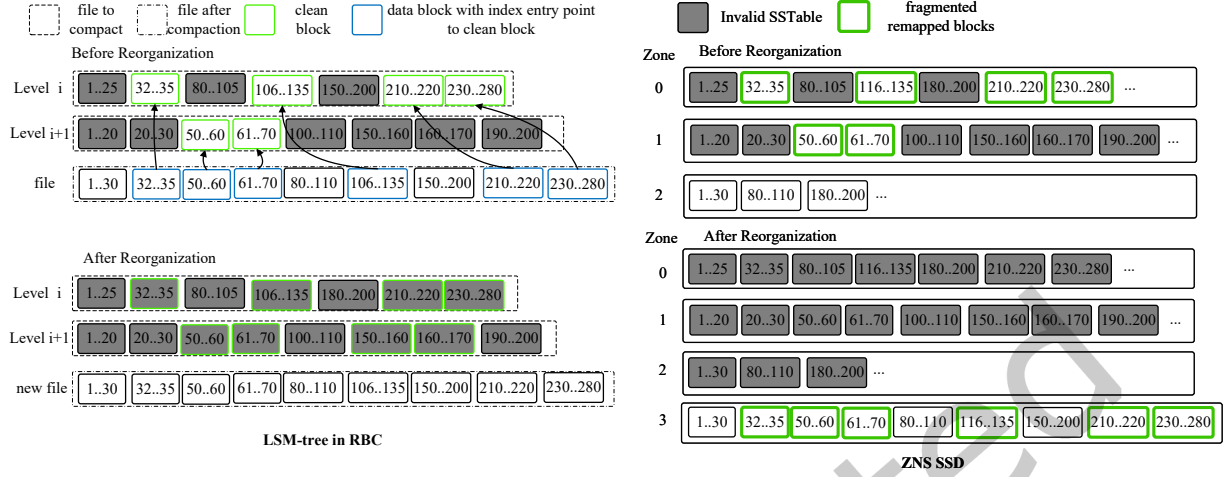


Fig. 15. RBC reorganizes fragmented clean blocks across LSM-tree levels and sequentially remaps them to ZNS zones.

remapping table exceeds a predefined memory threshold), RBC can selectively evict cold remapped entries back to their original logical mappings, trading a minor read amplification for bounded memory usage.

RBC's spatially guided prefetching cache coordination bridges the gap between write efficiency and read locality, which we further enhance with background reorganization.

3.2.1 Delaying Garbage Collection for Highly Fragmented Zones. RBC defers garbage collection for highly fragmented zones to reduce data migration and write amplification. When the system is under heavy I/O load, garbage collection for these zones is postponed. This approach reduces overhead and improves system responsiveness.

Motivation: Why Defer GC for Fragmented Zones. Highly fragmented zones cause excessive data movement during GC and increase write amplification. To address this, RBC delays GC in these zones when the system is under load.

Strategy Overview. RBC classifies zones based on two metrics: valid data block ratio and remapped block count. Zones with high fragmentation are assigned low reclamation priority.

Triggering Conditions. GC deferral is activated when:

- I/O queue length exceeds a threshold, or
- Read/write throughput exceeds a configurable peak percentage.

Execution During Idle Periods. When bandwidth drops below 30% of peak, background GC reclaims previously deferred zones. This idle-aware mechanism ensures low GC overhead and minimal disruption to user I/O.

3.2.2 Reorganization of Fragmented Data Blocks. RBC periodically identifies and reorganizes logically sequential but physically scattered clean blocks. These fragmented blocks are consolidated into physically contiguous layouts during idle periods to restore spatial locality and reduce GC overhead.

Scanning and Identifying Fragmented Data Blocks. The system periodically scans the storage space to evaluate the physical locations and logical order of each data block. Logically contiguous but physically scattered remapped clean data blocks are identified and marked as "fragmented data blocks." RBC analyzes files to calculate their degree of fragmentation.

Selection of Files for Reorganization. RBC prioritizes file reorganization based on the remapping ratio, from highest to lowest. RBC selects files with the highest read access frequency and the largest number of remapped clean blocks for reorganization.

Data Migration and Writing. RBC sequentially reads fragmented data blocks within a file and writes them to a new file, ensuring that the data blocks are stored contiguously on the physical storage, as shown in figure 15. RBC reorganizes logically fragmented but frequently accessed clean data blocks across LSM levels and remaps them into a new SSTable. This reorganization restores physical contiguity on ZNS SSDs by grouping these blocks into new zones during idle periods, improving read locality and reducing GC overhead. The figure illustrates block flow and layout changes before and after reorganization at both the LSM-tree and ZNS device levels.

During the data migration process, RBC ensures data consistency through a dual-write mechanism. Throughout the migration, data is stored simultaneously at both the original and target locations until the migration transaction is fully committed. This mechanism effectively guarantees that, even in the event of a migration failure, data can still be read from the original location, preventing any risk of data loss.

Preserving Zone-Level Data Placement. To avoid disrupting the zone-level lifespan management optimized by the original SSTable placement, RBC carefully considers data block lifetimes during reorganization. During block migration, RBC groups and writes logically contiguous blocks with similar access and modification patterns into the same target zone. If fragmented blocks originate from SSTables with distinct lifetime hints, RBC prioritizes preserving this distinction during reorganization, thereby maintaining effective zone-level wear leveling and garbage collection efficiency.

Idle Period Reorganization and High Load Period Optimization. To avoid introducing additional overhead during normal operations, data reorganization is scheduled during **idle periods**. When ZNS SSDs have available read/write bandwidth, data block reorganization is carried out during idle phases of block compaction or during low-load periods.

Reorganization Triggering Policy. Reorganization is triggered when remapped blocks exceed 30% and bandwidth usage is below 30%. This dual-condition design ensures that reorganization activities are infrequent and minimize interference with foreground operations.

Reorganization Overhead Analysis. Our evaluation shows that background reorganization consumes less than 5% of the total device bandwidth during peak compaction periods. Since reorganization is performed at low priority and is throttled based on available I/O capacity, its impact on user read/write latency is negligible.

3.2.3 Key Innovations. RBC introduces two techniques that enhance ZNS efficiency by balancing write throughput and read locality:

- **Deferred Garbage Collection:** Prioritizes zone reclamation based on fragmentation and system load to reduce GC overhead.
- **Idle-Time Block Reorganization:** Relocates logically sequential but physically scattered blocks via crash-consistent migration during idle periods.

3.3 Dynamic Compaction Read Strategy Based on Random Read Ratio

The data blocks read during block compaction are randomly distributed within files, leading to increased random reads. During compaction, if the reuse ratio of the file's data blocks is low, more random read operations are triggered, increasing latency. RBC introduces a dynamic file read strategy based on the ratio of random read key-value pairs. The strategy dynamically monitors the reuse ratio of data blocks within a file and adjusts the file reading method accordingly to optimize read latency during the compaction process.

Evaluating the Proportion of Non-Reused Data Blocks. During block compaction, RBC dynamically monitors the proportion of non-reused data blocks (i.e., dirty blocks) in each file to evaluate the random read ratio during block-grained compaction.

Dynamic Read Strategy Adjustment. In the dynamic read strategy, the dirty block threshold p is defined as the proportion of dirty blocks (i.e., blocks requiring merging and sorting) to the total number of data blocks in the file. This threshold determines the block compaction approach.

- **When the dirty block ratio is greater than p :** In this case, the majority of data blocks in the file require merging and sorting. Therefore, file-grained compaction is used.
- **When the dirty block ratio is between $p/2$ and p :** Ignoring clean blocks during reads may increase compaction read latency. To address this, RBC reads the entire file to reduce latency.
- **When the dirty block ratio is less than $p/2$:** RBC adopts a block compaction strategy, skipping clean block reads to optimize compression efficiency.

If the threshold p is set too high, it may lead to the need to handle a large number of dirty blocks during compaction, making the compaction more complex and increasing latency. Conversely, if p is set too low, the system may frequently choose block-grained compaction, which could result in higher random read overhead and degrade overall system performance. In this work, the threshold p is determined based on empirical values. Under typical load scenarios, p is set to 0.8, meaning that when the proportion of dirty blocks in a file exceeds 80%, the system switches to file-grained compaction.

Threshold Justification. We empirically set the dirty block threshold to $p = 0.8$, balancing prefetch locality and avoiding read amplification beyond this fragmentation point.

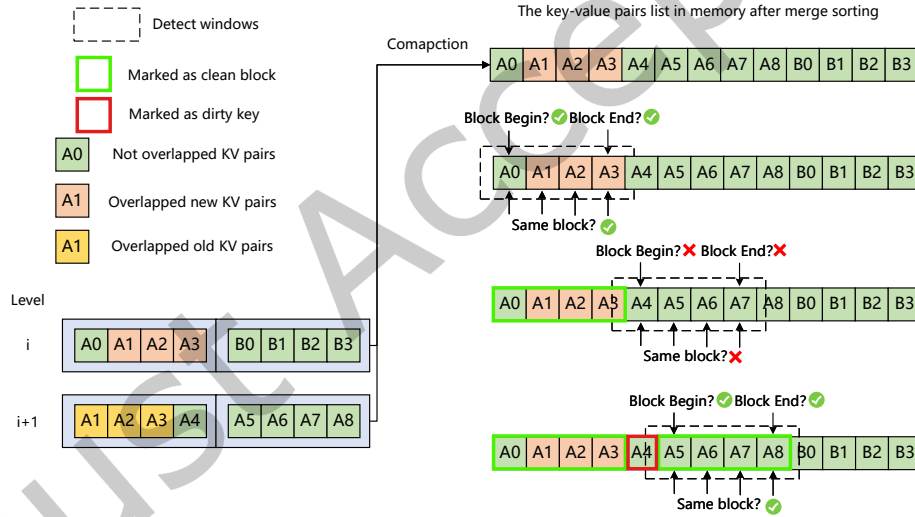


Fig. 16. Sliding Window Detection for Clean Blocks in Compaction Reads.

Threshold Selection. In RBC, we empirically set the dirty block ratio threshold to $p = 0.8$. This selection is motivated by the trade-off between maintaining sequential prefetching efficiency and avoiding read amplification. When over 80% of a region's blocks are clean, prefetching remains highly beneficial; conversely, when the fraction of dirty blocks exceeds 20%, the probability of fragmented reads and cache pollution increases significantly. Our internal microbenchmarks confirm that prefetch hit rates sharply decline beyond this threshold, justifying the choice of $p = 0.8$ as a practical balance.

Adaptive Threshold Adjustment. We acknowledge that different workloads may exhibit varying levels of fragmentation and access patterns. In future work, RBC could incorporate an adaptive threshold adjustment

mechanism, dynamically tuning p based on real-time fragmentation metrics (e.g., observed sequentiality ratios or dirty block clustering density) to further optimize read performance across diverse workload characteristics.

Sliding Window Algorithm for Optimized Reads. When the dirty block ratio is between $p/2$ and p , RBC uses a sliding window to detect clean and dirty blocks during file reads, avoiding the additional overhead of separately traversing the file to identify clean blocks, as shown in figure 16.

Window Initialization: The sliding window starts at the beginning of the new SSTable, with a fixed size of block (aligned with block granularity). **Block Scanning & Condition Verification:** For each window position, three critical conditions are sequentially verified:

- (1) **Condition 1 (Block Integrity):** All KV pairs in the window must belong to the same original data block in old SSTables.
 - **Validation:** Check metadata tags (e.g., `block_id`) of KV pairs for consistency.
- (2) **Condition 2 (Start Boundary Alignment):** The first KV pair in the window must match the first KV pair of the original data block.
 - **Validation:** Compare the offset of the first KV pair with the original block's starting address.
- (3) **Condition 3 (End Boundary Alignment):** The last KV pair in the window must match the last KV pair of the original data block.
 - **Validation:** Compare the end offset of the last KV pair with the original block's termination address.

Redundancy Determination & Handling:

- If all conditions are satisfied, mark all KV pairs in the window as reusable (clean block), directly referencing the original block's physical address to avoid redundant writes.
- If any condition fails, shift the window forward by one KV pair and repeat step 2.

Optimization Design of Sliding Window Algorithm

- **Low Computational Overhead:** Using a sliding window to check cleanblock during the compaction to avoid the additional traversal overhead.
- **Spatial Locality Preservation:** Window sliding step aligns with block granularity (4KB), ensuring detection consistency with physical storage layout.
- **Metadata Assistance:** Utilizes existing KV pair metadata (`block_id`, offsets) without requiring additional fingerprint indexes.

4 Evaluation

We evaluated RBC on a 12-core Intel Xeon server with 16GB RAM and a 12TB ZNS SSD using ZenFS under Ubuntu 16.04. Benchmarking compared RBC against RocksDB, DepotDB[13], and traditional block compaction. RBC operates independently of specific device or file system configurations, relying only on standard ZNS semantics.

All systems used WAL and disabled compression. RBC and block compaction applied 4KB granularity for dirty block detection. RBC switches to file-level compaction when overlapping KV pairs exceed 80%. As shown in Table1, SSTable size was 64MB, with 4 compaction threads, 2 flush threads, and 20MB cache. YCSB workloads used 512B KV pairs (16B keys) with Zipfian distribution ($\text{Zipf} = 0.99$).

4.1 YCSB Core Workloads

We compared RBC, block compaction, RocksDB, and LSM-tree+ across six YCSB workloads (A–F), covering different read/write mixes and scan intensity: A (50% Read, 50% Write), B (95% Read, 5% Write), C (100% Read), D (95% Read, 5% Write), E (95% Scan, 5% Write), and F (50% Read, 50% Read-Modify-Write). As shown in Figure 17a, RBC consistently achieves the highest throughput, particularly excelling in mixed (A, F) and scan-heavy (E)

Configuration Item	Parameter Value
Dirty Block Detection Granularity	4KB
Dirty Block Threshold For Dynamic Read	80%
SSTable Size	64MB
Background Compaction Threads	4
Background Flush Threads	2

Table 1. Configuration Parameters

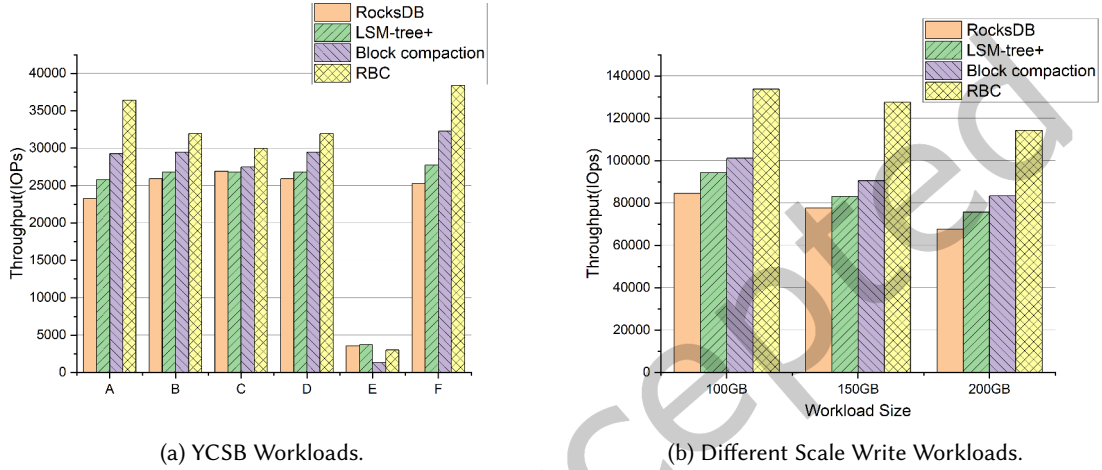


Fig. 17. Throughput under YCSB and Different Scale Workloads

scenarios, reaching up to 40K IOPS. These results confirm RBC’s balanced performance across diverse workload patterns.

4.2 Throughput under Write-Intensive Workloads

We loaded the data into the system using sequential writes, followed by random write operations, to evaluate the throughput of RBC, RocksDB, and traditional block compaction under random write conditions. Our test covered datasets ranging from 100GB to 200GB.

Write Amplification Measurement Methodology. We separately measure write amplification at two levels:

- **LSM-tree Level WA:** We calculate LSM-tree write amplification as the ratio of total bytes written by user-level compaction operations (including flushes and compactions across all levels) to the total bytes of user writes ingested into the system. We collect the cumulative compaction bytes using RocksDB’s internal statistics counters (`bytes.compacted.add`, `bytes.compacted.remove`) during the workload runtime.
- **GC Level WA:** We measure garbage collection (GC) level write amplification based on ZNS SSD internal statistics. Specifically, we track the total physical bytes written to the device (via the SSD firmware’s write counters) and subtract the amount of user data bytes issued. GC WA is then computed as the additional writes caused purely by zone resets and data migrations divided by user data size.
- **Total WA:** The overall system write amplification is the sum of LSM-tree WA and GC WA.

All WA measurements are taken during steady-state operation, after the database reaches a stable size and compaction has converged. We repeat all experiments three times and report the average values to ensure statistical reliability.

The experimental results in figure 17b show that under write-intensive workloads, RBC outperforms RocksDB and traditional block compaction methods by up to 69% and 37%, respectively. This improvement is attributed to RBC’s low-overhead strategy for detecting non-overlapping key-value pairs, which effectively reduces write amplification and lowers read latency through prefetch cache optimization.

4.3 WA under Write-Intensive Workloads

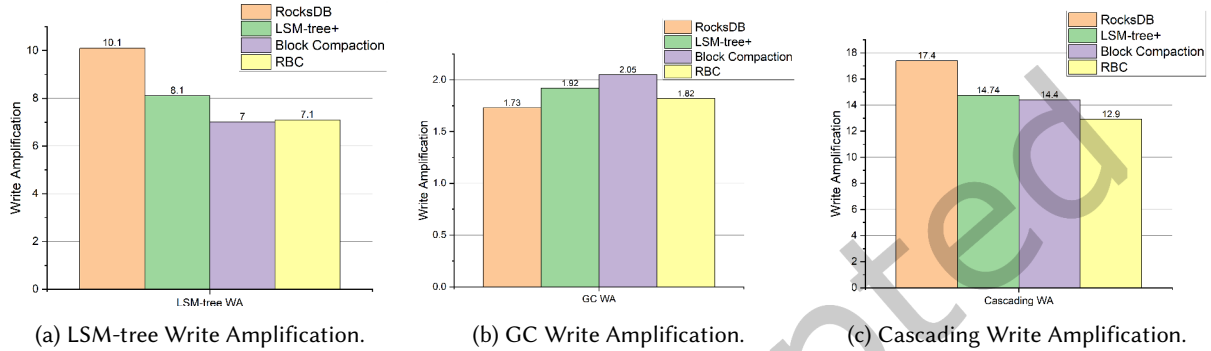


Fig. 18. Write Amplification under Write-Intensive Workloads

We evaluated the write amplification of RBC, RocksDB, and block compaction under random write conditions by first loading data into the system through sequential writes, followed by random write operations. The experimental results figure 18 show that, in the Cascading Write Amplification, RBC reduced write amplification by 25.9% compared to RocksDB and 10.4% compared to block compaction. RBC reorganizes remapped data blocks during idle periods. RBC also defers the reclamation of highly fragmented zones. These approaches effectively mitigate the impact of fragmentation on write amplification.

4.4 Performance Under Real-World Facebook Workloads

In this workload, the generated keys exhibit a **prefix locality pattern** using a Prefix_dist model: keys sharing the same prefix are clustered together logically but are physically randomly distributed upon insertion. This configuration simulates a realistic operational scenario where insert-heavy applications (e.g., logging, telemetry ingestion) maintain some logical grouping (e.g., by tenant ID or topic) while suffering from physical fragmentation at the storage layer. We simulate a random insertion workload consisting of 50 million key-value pairs. We evaluate RBC under this Facebook RocksDB-based synthetic benchmarks. Figure 19a shows the throughput (measured in IOPS) across the different systems. RBC achieves the highest throughput at 65,749 IOPS, significantly outperforming the other designs:

- Compared to RocksDB (45,435 IOPS), RBC improves throughput by **44.8%**.
- Compared to LSM-tree+ (51,664 IOPS), RBC achieves a **27.3%** improvement.
- Compared to traditional Block Compaction (53,914 IOPS), RBC provides a **21.2%** gain.

4.5 Throughput for Different Key-Value Pair Sizes

To further investigate the impact of different key-value pair sizes on system throughput, we fixed all other parameters and tested key-value pairs ranging from 64B to 1KB in size.

The figure 19b show that as the key-value pair size increases, the throughput of all key-value storage systems improves. RBC achieves the highest throughput across all key-value sizes, outperforming Block Compaction

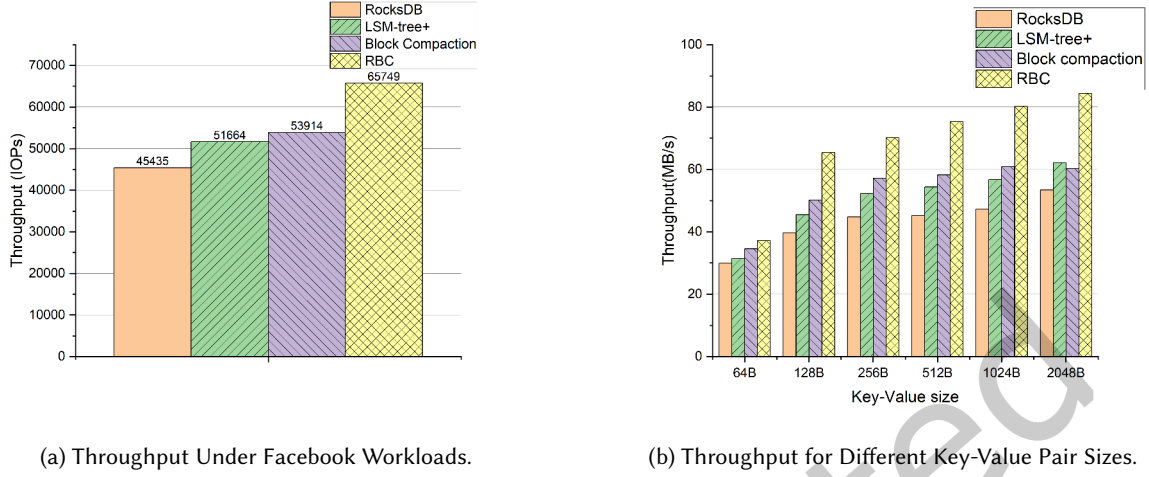


Fig. 19. Throughput under Facebook Workloads and Different Key-Value Pair Sizes

by 7.5%–39%, LSM-tree+ by 17%–35%, and RocksDB by 23%–51%, with the performance advantage becoming particularly pronounced at larger sizes (1024B–2048B).

During this process, RBC consistently outperforms other systems. However, as the key-value pair size increases, the relative throughput improvement of RBC gradually decreases. This trend is attributed to RBC’s efficient utilization of SSD bandwidth, which ensures that throughput increases steadily with the key-value pair size.

4.6 Throughput under Mixed Workloads

We used YCSB, the industry-standard benchmark for key-value stores, to evaluate the overall performance under mixed read-write workloads. First, we randomly loaded 100 million key-value pairs and executed a workload consisting of 100 million operations with varying read-write ratios (30:70, 50:50, 70:30).

As shown in figure 20a:

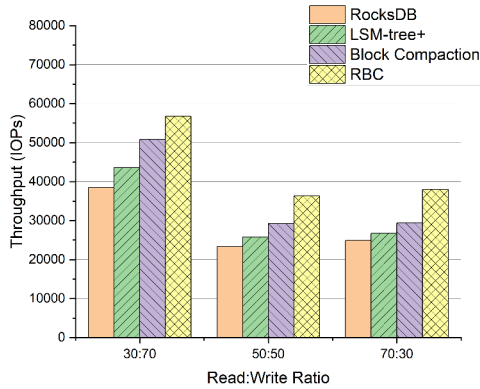
- **Write-Intensive (30:70):** RBC shows 27.8%–47.2% higher throughput compared to alternative systems, with the most significant advantage over baseline RocksDB implementations.
- **Balanced (50:50):** RBC maintains 24.4%–44.0% performance improvements, particularly excelling in mixed read-write scenarios where traditional systems typically struggle.
- **Read-Intensive (70:30):** RBC shows 28.8%–52.3% better throughput, proving its effectiveness even in read-dominated workloads.

The evaluation reveals RBC’s strongest relative advantage in write-heavy scenarios, while maintaining substantial lead across all workload types. Notably, RBC consistently outperforms the next-best system (Block Compaction) by at least 24% in every configuration, demonstrating its balanced architectural advantages for both read and write operations.

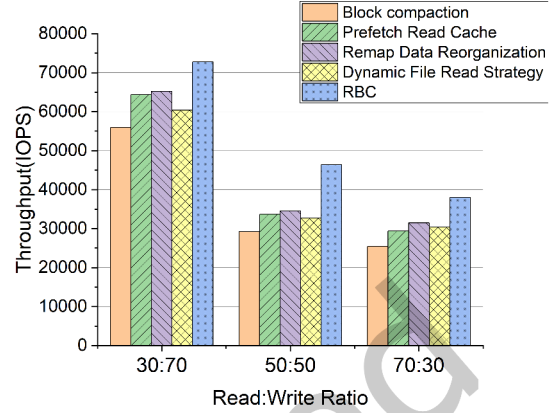
4.7 Individual Effects of Each Component

From the fig20b, it can be seen that as each module of RBC is progressively introduced, the system performance (measured by throughput) shows a significant improvement.

RBC: RBC demonstrates consistent performance improvements across all workload profiles, with throughput enhancements ranging from 1.30x (30:70) to 1.59x (50:50) over Block Compaction. This efficiency is driven



(a) Throughput Under Different Read-Write Ratios.



(b) Throughput of Each Component Under Different Read-Write Ratios.

Fig. 20. Throughput under Different Read-Write Ratios and Throughput of Each Component Under Different Read-Write Ratio

by RBC's optimal handling of both read and write operations, enabling improved throughput, particularly in write-heavy and mixed workloads.

Prefetch Read Cache:

The prefetch cache mechanism shows moderate improvements in throughput, with gains of 1.15x to 1.16x over Block Compaction, depending on the workload. By leveraging efficient prefetching, it enhances the hit rate of cached data blocks, thereby reducing the latency associated with random reads in mixed and read-heavy workloads.

Remap Data Reorganization Strategy:

Building upon the prefetch cache, the reorganization strategy delivers incremental throughput improvements ranging from 1.17x (30:70) to 1.24x (70:30), showing particular effectiveness in read-heavy environments. The reorganization of remapped data blocks has minimized storage fragmentation, leading to a decrease in the frequency of random read I/O operations, especially in read-intensive scenarios.

Dynamic File Read Strategy:

The dynamic approach exhibits workload-adaptive characteristics, with its most significant throughput improvement of 1.19x over Block Compaction observed in read-heavy (70:30) workloads. By adjusting read strategies based on the proportion of random reads, it minimizes the latency of reading data blocks, reducing delays caused by random access patterns in highly variable workloads.

4.8 Throughput under Range Queries Workloads

We also evaluated the overall performance of RBC, block compaction, and RocksDB under range query workloads. The tests included short-range queries (reading sequences of 100 key-value pairs) and long-range queries (reading sequences of 1000 key-value pairs).

As figure 21 shows, for short and long-range queries, RBC improves performance by 1.15x and 1.75x respectively, due to restored locality through remapped block reorganization.

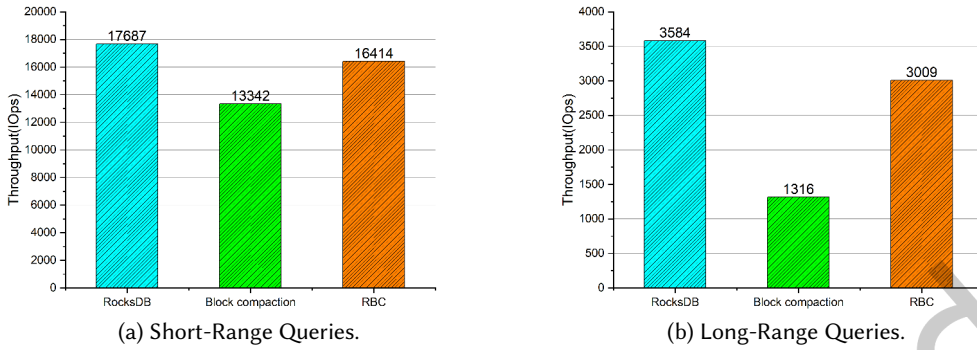


Fig. 21. Throughput for Short-Range Queries and Long-Range Queries

4.9 Background Overhead Analysis

To assess RBC's background overhead, we analyze compaction I/O, CPU usage, cache efficiency, and memory usage using YCSB Workload A (100GB dataset).

Figure 22a shows RBC reduces background I/O volume (506GB write, 387GB read), significantly lower than RocksDB (763GB/513GB), indicating reduced write amplification via optimized data layout.

Figure 22c shows compaction dominates background CPU time (93.09%), while RBC-specific components (remap and prefetch) remain below 6%, confirming their low overhead.

Figure 22b demonstrates high prefetch cache hit rates (75.6%–78.3%), close to RocksDB's peak, validating RBC's ability to preserve locality.

Figure 22d shows remap table memory grows linearly (46MB to 184MB for 100GB–400GB datasets), staying under 0.2% of total data, confirming RBC's lightweight design.

5 Related Work

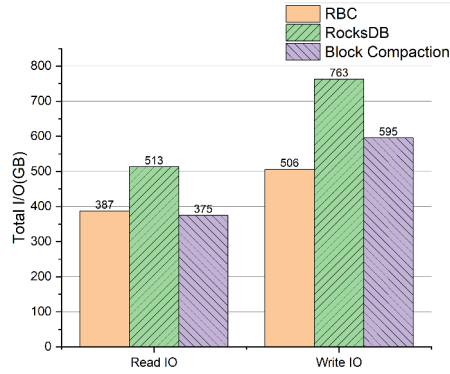
5.1 ZNS-Aware KV Store Designs

ZoneKV [18] is a key-value storage system designed for ZNS SSDs. It introduces a lifecycle-aware zone model and level-aligned zone allocation to reduce space amplification and intra-zone fragmentation. While ZoneKV improves spatial layout and zone efficiency, it does not address read inefficiencies introduced by block remapping and logical discontinuity.

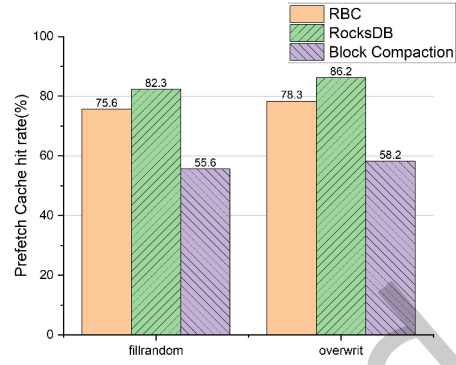
eZNS [20] proposes an elastic zone interface that dynamically adapts zone resources based on workload behavior. It introduces a zone arbiter and hierarchical I/O scheduler to improve throughput and latency. However, eZNS operates at the device interface level and does not address compaction strategies within the storage engine.

LL-Compaction [16] introduces lifetime-leveling compaction for ZNS SSDs. It assigns SSTables with similar update frequencies to the same zone to mitigate fragmentation and avoid host-side GC. This reduces space amplification and improves I/O performance without triggering additional write overhead. In contrast, RBC focuses on reducing read amplification and logical fragmentation by introducing remapping-aware background reorganization and spatial prefetch coordination, which can complement LL-Compaction's lifetime-aligned write optimization.

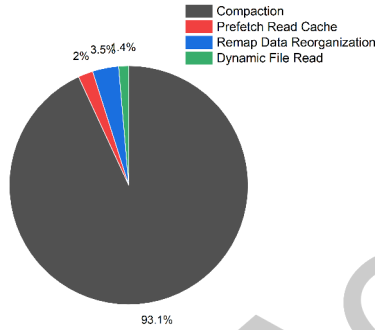
RBC complements both ZoneKV and eZNS by focusing on LSM-tree compaction logic and read path optimizations. It directly addresses block-level remapping and read amplification through remapping-aware prefetch and background reorganization.



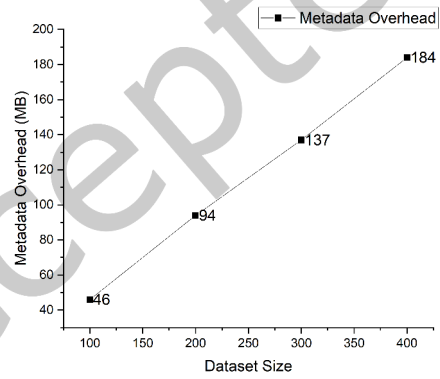
(a) Compaction and Reorganization I/O.



(b) Prefetch Cache Efficiency.



(c) CPU time breakdown of background operations in RBC.



(d) Thread and Memory Usage.

Fig. 22. Background Overhead Analysis.

RBC introduces a randomness-resistant compaction strategy for ZNS SSDs, integrating remapping-aware prefetching, background reorganization, and adaptive file reading. Evaluations show RBC significantly improves throughput and reduces read latency across diverse workloads.

5.2 Block-Level Compaction Techniques

Prior work on block-grained compaction, such as [26], has explored selective rewriting of dirty blocks to reduce write amplification. Systems like X-Engine and KVSSD also support block-level data organization for higher write throughput. However, these approaches primarily focus on write path efficiency and often omit coordinated read path optimization or metadata-aware reorganization. In contrast, RBC integrates remap-aware prefetching, dynamic metadata tracking, and background reorganization to address the spatial fragmentation and read inefficiencies that emerge in ZNS environments. This coordinated design distinguishes RBC as a holistic block-granular solution.

LavaStore [25] is a production-grade RocksDB variant optimized for write-intensive workloads at ByteDance. It features a KV-separation model, a WAL-specialized engine type, and a customized user-space append-only file system. While it significantly reduces read/write latency, it does not explicitly address block-level remapping fragmentation or cache prefetch inefficiency.

PhatKV [28] targets metadata-intensive workloads and proposes a two-level Piece Hash Index and a Three-Stage Aggregation strategy to eliminate sorting overhead and improve single-key lookup performance. It achieves significant throughput gains but does not address LSM-tree compaction or ZNS-specific data layout challenges.

HyperDB [29] is a hybrid KV store optimized for heterogeneous SSD deployments (NVMe + SATA). It deploys zone-based data layout on NVMe and semi-sorted tables on SATA, and introduces preemptive block-granularity compaction to reduce merge overhead. However, it focuses primarily on media placement and hybrid I/O efficiency, and does not coordinate remapping with read path optimization.

X-Engine [15] is an optimized LSM-tree KV engine widely deployed in WeChat, featuring a block-level compaction mechanism with KV separation to reduce write amplification. It integrates prefix bloom filters and log-structured block organization to maintain high query performance. However, X-Engine does not address physical fragmentation or spatial locality restoration, which are critical for ZNS SSDs.

Similarly, **KVSSD** [27] employs device-level block management with host-visible key-value abstractions. Although its design enables efficient block-level operations, it lacks host-layer coordination for remapping-aware compaction or prefetch-guided reads. RBC builds on these ideas by explicitly managing remap tables and background reorganization to reduce read amplification and restore contiguity in fragmented zones.

In contrast, **RBC introduces a remapping-aware compaction model** with three key differentiators:

- A scalable remapping table enabling block-level read prediction;
- A remap-aware spatially guided prefetch cache that recovers spatial locality;
- Background reorganization to restore physical contiguity in fragmented SSTables.

5.3 Summary

Existing systems focus on either improving write efficiency or managing device-level zoning, but few address the read latency implications of block-level remapping. RBC uniquely bridges this gap by combining block-level write optimization with prefetch-coordinated read path design, making it particularly suited for performance-critical ZNS deployments.

6 Conclusion

We propose Randomness-Resistant Block-grained Compaction (RBC) to reduce CPU overhead and read latency in ZNS SSDs. RBC introduces a remapping table to guide prefetching, defers reclamation of fragmented zones, and dynamically adjusts read strategies based on block reuse ratios.

Evaluations show RBC improves throughput by up to 69% over RocksDB and 37% over state-of-the-art block-level schemes. It consistently outperforms baselines under both YCSB and Facebook workloads, demonstrating robust real-world effectiveness. Future work includes distributed remap coordination, ML-based reorganization scheduling, and integration with storage-class memory.

Acknowledgment

This work was supported by the NSFC under Grant No.62172178 and No.61821003, as well as by Alibaba Group through Alibaba Innovative Research (AIR) Program.

7 Citations

References

- [1] Manos Athanassoulis and Stratos Idreos. 2016. Design Tradeoffs of Data Access Methods. In *Proceedings of the 2016 International Conference on Management of Data, SIGMOD Conference 2016, San Francisco, CA, USA, June 26 - July 01, 2016*, Fatma Özcan, Georgia Koutrika, and Sam Madden (Eds.). ACM, 2195–2200. doi:10.1145/2882903.2912569
- [2] Manos Athanassoulis, Michael S. Kester, Lukas M. Maas, Radu Stoica, Stratos Idreos, Anastasia Ailamaki, and Mark Callaghan. 2016. Designing Access Methods: The RUM Conjecture. In *Proceedings of the 19th International Conference on Extending Database Technology, EDBT 2016, Bordeaux, France, March 15-16, 2016, Bordeaux, France, March 15-16, 2016*, Evangelia Pitoura, Sofian Maabout, Georgia Koutrika, Amélie Marian, Letizia Tanca, Ioana Manolescu, and Kostas Stefanidis (Eds.). OpenProceedings.org, 461–466. doi:10.5441/002/edbt.2016.42
- [3] Oana Balmau, Florin Dinu, Willy Zwaenepoel, Karan Gupta, Ravishankar Chandhiramoorthi, and Diego Didona. 2018. SILK+ Preventing Latency Spikes in Log-Structured Merge Key-Value Stores Running Heterogeneous Workloads. *ACM Trans. Comput. Syst.* 36, 4 (2018), 12:1–12:27. doi:10.1145/3380905
- [4] Oana Balmau, Florin Dinu, Willy Zwaenepoel, Karan Gupta, Ravishankar Chandhiramoorthi, and Diego Didona. 2019. SILK: Preventing Latency Spikes in Log-Structured Merge Key-Value Stores. In *2019 USENIX Annual Technical Conference, USENIX ATC 2019, Renton, WA, USA, July 10-12, 2019*, Dahlia Malkhi and Dan Tsafir (Eds.). USENIX Association, 753–766. <https://www.usenix.org/conference/atc19/presentation/balmau>
- [5] Matias Björling, Abutalib Aghayev, Hans Holmberg, Aravind Ramesh, Damien Le Moal, Gregory R. Ganger, and George Amvrosiadis. 2021. ZNS: Avoiding the Block Interface Tax for Flash-based SSDs. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 689–703. <https://www.usenix.org/conference/atc21/presentation/bjorling>
- [6] Matias Björling. 2020. NVMe Express Zoned Namespaces Command Set 1.0. Available from <http://www.nvmexpress.org/specifications>.
- [7] Dhruva Borthakur. 2014. RocksDB: A persistent key-value store. *Facebook, Inc., Menlo Park, CA, USA, Tech. Rep* (2014).
- [8] Yunpeng Chai, Yanfeng Chai, Xin Wang, Haocheng Wei, Ning Bao, and Yushi Liang. 2019. LDC: A Lower-Level Driven Compaction Method to Optimize SSD-Oriented Key-Value Stores. In *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*. IEEE, 722–733. doi:10.1109/ICDE.2019.00070
- [9] Helen HW Chan, Chieh-Jan Mike Liang, Yongkun Li, Wenjia He, Patrick PC Lee, Lianjie Zhu, Yaozu Dong, Yinlong Xu, Yu Xu, Jin Jiang, et al. 2018. Hashkv: Enabling efficient updates in {KV} storage via hashing. In *2018 {USENIX} Annual Technical Conference ({USENIX}) {ATC} 18*, 1007–1019.
- [10] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. 2017. Monkey: Optimal Navigable Key-Value Store. In *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017*, Semih Salihoglu, Wenchao Zhou, Rada Chirkova, Jun Yang, and Dan Suciu (Eds.). ACM, 79–94. doi:10.1145/3035918.3064054
- [11] Niv Dayan and Stratos Idreos. 2018. Dostoevsky: Better Space-Time Trade-Offs for LSM-Tree Based Key-Value Stores via Adaptive Removal of Superfluous Merging. In *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018*, Gautam Das, Christopher M. Jermaine, and Philip A. Bernstein (Eds.). ACM, 505–520. doi:10.1145/3183713.3196927
- [12] Facebook. [n. d.]. RocksDB. <https://rocksdb.org>.
- [13] Zhixin Fan, Yaohui Zhang, Jiang Zhou, Bo Li, Yong Chen, and Weiping Wang. 2023. DepotDB: Mitigate Write Amplification with Region Compaction on KV Stores for Periodic Write-Intensive Workloads. In *IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking, ISPA/BDCloud/SocialCom/SustainCom, 2023, Wuhan, China, December 21-24, 2023*. IEEE, 561–566. doi:10.1109/ISPA-BDCloud-SOICALCOM-SUSTAINCOM59178.2023.00105
- [14] Sanjay Ghemawat and Jeff Dean. 2011. LevelDB.
- [15] Gui Huang, Xuntao Cheng, Jianying Wang, Yujie Wang, Dengcheng He, Tieying Zhang, Feifei Li, Sheng Wang, Wei Cao, and Qiang Li. 2019. X-Engine: An optimized storage engine for large-scale E-commerce transaction processing. In *Proceedings of the 2019 International Conference on Management of Data*. 651–665.
- [16] Jeeyoon Jung and Dongkun Shin. 2022. Lifetime-leveling LSM-tree compaction for ZNS SSD. In *Proceedings of the 14th ACM Workshop on Hot Topics in Storage and File Systems (Virtual Event) (HotStorage '22)*. Association for Computing Machinery, New York, NY, USA, 100–105. doi:10.1145/3538643.3539741
- [17] Lanyue Lu, Thanumalayan Sankaranarayanan Pillai, Hariharan Gopalakrishnan, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. 2017. Wiskey: Separating keys from values in ssd-conscious storage. *ACM Transactions on Storage (TOS)* 13, 1 (2017), 1–28.
- [18] Mingchen Lu, Peiquan Jin, Xiaoliang Wang, Yongping Luo, and Kuankuan Guo. 2023. ZoneKV: A Space-Efficient Key-Value Store for ZNS SSDs. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. doi:10.1109/DAC56929.2023.10247926
- [19] Chen Luo and Michael J. Carey. 2020. LSM-based storage techniques: a survey. *VLDB J.* 29, 1 (2020), 393–418. doi:10.1007/s00778-019-00555-y

- [20] Jaehong Min, Chenxingyu Zhao, Ming Liu, and Arvind Krishnamurthy. 2023. eZNS: An Elastic Zoned Namespace for Commodity ZNS SSDs. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 461–477. <https://www.usenix.org/conference/osdi23/presentation/min>
- [21] Fengfeng Pan, Yinliang Yue, and Jin Xiong. 2017. dCompaction: Delayed compaction for the LSM-tree. *International Journal of Parallel Programming* 45, 6 (2017), 1310–1325.
- [22] Pandian Raju, Rohan Kadekodi, Vijay Chidambaram, and Ittai Abraham. 2017. Pebblesdb: Building key-value stores using fragmented log-structured merge trees. In *Proceedings of the 26th Symposium on Operating Systems Principles*. 497–514.
- [23] S. Ghemawat and J. Dean. [n. d.]. LevelDB. <https://leveldb.org>.
- [24] Subhadeep Sarkar, Dimitris Staratzis, Zichen Zhu, and Manos Athanassoulis. 2022. Constructing and analyzing the LSM compaction design space (updated version). *arXiv preprint arXiv:2202.04522* (2022).
- [25] Hao Wang, Jiaxin Ou, Ming Zhao, Sheng Qiu, Yizheng Jiao, Yi Wang, Qizhong Mao, Zhengyu Yang, Yang Liu, Jianshun Zhang, et al. 2024. LavaStore: ByteDance’s Purpose-Built, High-Performance, Cost-Effective Local Storage Engine for Cloud Services. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3799–3812.
- [26] Xiaoliang Wang, Peiquan Jin, Bei Hua, Hai Long, and Wei Huang. 2022. Reducing Write Amplification of LSM-Tree with Block-Grained Compaction. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 3119–3131. doi:10.1109/ICDE53745.2022.00279
- [27] Sung-Ming Wu, Kai-Hsiang Lin, and Li-Pin Chang. 2018. KVSSD: Close integration of LSM trees and flash translation layer for write-efficient KV store. In *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 563–568.
- [28] Yiwen Zhang, Guokuan Li, Kai Lu, Jiguang Wan, Ting Yao, Huatao Wu, and Daohui Wang. [n. d.]. PhatKV: Towards an Efficient Metadata Engine for KV-based File Systems on Modern SSD. ([n. d.]).
- [29] Ruisong Zhou, Yuzhan Zhang, Chunhua Li, Ke Zhou, Peng Wang, Gong Zhang, Ji Zhang, and Guangyu Zhang. 2024. HyperDB: a Novel Key Value Store for Reducing Background Traffic in Heterogeneous SSD Storage. In *Proceedings of the 53rd International Conference on Parallel Processing*. 453–463.

Received 4 January 2025; revised 17 July 2025; accepted 15 August 2025